

Matrice d'applicabilité des champs par type de serveur

Quels champs sont collectés / nullable-N/A / absents / différents selon l'OS et le système de fichiers. C'est le [rulebook du playbook](#) (complète la spec).

Matrice d'applicabilite des champs

Legende des cellules : **collecte** = champ renseigne avec une vraie valeur ; **nullable (N/A)** = champ present dans le document mais a `null` car non applicable a cette combinaison OS/fstype ; **absent** = clef non emise (niveau de topologie vide) ; **different** = champ renseigne mais via une autre source/semantique. La commande source est indiquee entre parentheses.

Champ	AIX (jfs2)	RHEL/Oracle (xfs)	Debian/Ubuntu (ext4)	SUSE/SLES (btrfs)	Linux sans LVM	Windows (NTFS/ReFS)
host_id, hostname, gsma_code, host_role_env	collecte (CMDB / hostname , uname -n)	collecte (CMDB / hostname)	collecte (CMDB / hostname)	collecte (CMDB / hostname)	collecte (CMDB / hostname)	collecte (CMDB / hostname , \$env:COMPUTERNAME)
platform	collecte = aix (uname -s)	collecte = linux (uname -s)	collecte = linux (uname -s)	collecte = linux (uname -s)	collecte = linux (uname -s)	collecte = windows (\$PSVersionTable / WinRM)
os_family	collecte = AIX (oslevel)	collecte = RedHat/OracleLinux (/etc/os-release)	collecte = Debian (/etc/os-release)	collecte = Suse (/etc/os-release)	collecte (/etc/os-release)	collecte = Windows (Get-CimInstance Win32_OperatingSystem)
os_distribution	collecte = AIX (oslevel -s)	collecte = RHEL/OracleLinux (/etc/os-release)	collecte = Ubuntu/Debian (/etc/os-release)	collecte = SLES (/etc/os-release)	collecte (/etc/os-release)	collecte = WindowsServer (Win32_OperatingSystem.Caption)
os_version, architecture	collecte (oslevel -s , prtconf / uname -p)	collecte (/etc/os-release , uname -m)	collecte (/etc/os-release , uname -m)	collecte (/etc/os-release , uname -m)	collecte (/etc/os-release , uname -m)	collecte (Win32_OperatingSystem.Version , \$env:PROCESSOR_ARCHITECTURE)

Champ	AIX (jfs2)	RHEL/Oracle (xfs)	Debian/Ubuntu (ext4)	SUSE/SLES (btrfs)	Linux sans LVM	Windows (NTFS/ReFS)
snapshot_id, scan_timestamp, collection_method, scan_confidence	collecte (metadonnees pipeline)	collecte (metadonnees pipeline)	collecte (metadonnees pipeline)	collecte (metadonnees pipeline)	collecte (metadonnees pipeline)	collecte (metadonnees pipeline, collection_method=winrm-powershell)
<i>filesystems[]</i>						
mount_point	collecte (df -k , lsfs)	collecte (df -k , findmnt)	collecte (df -k , findmnt)	collecte (findmnt , btrfs filesystem show)	collecte (df -k , findmnt)	different = lettre de lecteur / point de montage dossier (Get-Volume , Get-Partition)
label	collecte (lslv , lsfs)	collecte (blkid , xfs_admin -l)	collecte (blkid , e2label)	collecte (blkid , btrfs filesystem label)	collecte (blkid)	collecte (Get-Volume.FileSystemLabel)
fstype	collecte = jfs2 (lsfs -c)	collecte = xfs (findmnt -no FSTYPE)	collecte = ext4 (findmnt -no FSTYPE)	collecte = btrfs (findmnt -no FSTYPE)	collecte (findmnt -no FSTYPE)	collecte = ntfs/refs (Get-Volume.FileSystem)
size_total_bytes, size_used_bytes, size_available_bytes	collecte (df -k × 1024)	collecte (df -B1 / findmnt -b)	collecte (df -B1 / findmnt -b)	different = via btrfs filesystem usage / qgroups (df trompeur a cause des sous-volumes)	collecte (df -B1 / findmnt -b)	collecte = octets natifs (Get-Volume Size/SizeRemaining , used = Size - SizeRemaining)
reserved_bytes	nullable (N/A) — jfs2 sans blocs reserves	nullable (N/A) — xfs sans blocs reserves	collecte (tune2fs -l : Reserved block count × Block size)	nullable (N/A) — btrfs sans blocs reserves	collecte si ext4 (tune2fs -l) / nullable si xfs	nullable (N/A) — NTFS/ReFS sans blocs reserves
inodes_total, inodes_used	collecte (df -v ou df -i)	collecte (df -i , xfs_info)	collecte (df -i)	collecte (df -i ; partage dynamiquement entre sous-volumes)	collecte (df -i)	nullable (N/A) — pas d'inodes sur NTFS/ReFS
access_model	collecte = posix-mount-options	collecte = posix-mount-options	collecte = posix-mount-options	collecte = posix-mount-options	collecte = posix-mount-options	collecte = ntfs-acl
mount_options	collecte (mount , lsfs)	collecte (findmnt -no OPTIONS)	collecte (findmnt -no OPTIONS)	collecte (findmnt -no OPTIONS , options de sous-volume)	collecte (findmnt -no OPTIONS)	different = resume ACL non sensible via Get-Acl (champ mount_options laisse a null)

Champ	AIX (jfs2)	RHEL/Oracle (xfs)	Debian/Ubuntu (ext4)	SUSE/SLES (btrfs)	Linux sans LVM	Windows (NTFS/ReFS)
acl_summary	nullable (N/A) — modele POSIX	nullable (N/A) — modele POSIX	nullable (N/A) — modele POSIX	nullable (N/A) — modele POSIX	nullable (N/A) — modele POSIX	collecte = resume ACL non sensible (<code>Get-Acl</code>)
backing_kind	collecte = lvm-lv (<code>lslv</code> , <code>lsvg -l</code>)	collecte = lvm-lv partition (<code>lsblk -o TYPE</code> , <code>lvs</code>)	collecte = lvm-lv partition (<code>lsblk -o TYPE</code> , <code>lvs</code>)	collecte = lvm-lv partition btrfs (<code>lsblk -o TYPE</code>)	collecte = partition virtual-disk (<code>lsblk -o TYPE</code>)	collecte = virtual-disk partition storage-space (<code>Get-Partition</code> , <code>Get-VirtualDisk</code>)
backing_ref, group_ref	collecte (nom LV / rootvg-datavg, <code>lsvg -l</code>)	collecte (nom LV / VG, <code>lvs -o lv_name,vg_name</code>)	collecte (nom LV / VG, <code>lvs</code>)	collecte (nom LV/VG ou device btrfs)	collecte = backing_ref (device) ; group_ref nullable (pas de VG)	collecte (<code>Get-Partition.DiskNumber</code> , ref pool/virtual-disk)
content_aggregates	collecte (agregats 2e vague non sensibles)	collecte (agregats 2e vague)	collecte (agregats 2e vague)	collecte (agregats 2e vague)	collecte (agregats 2e vague)	collecte (agregats 2e vague)
<i>storage_topology.groups[]</i>						
groups[] (niveau)	collecte = lvm-vg (<code>lsvg</code>)	collecte si LVM = lvm-vg (<code>vgs</code>)	collecte si LVM = lvm-vg (<code>vgs</code>)	collecte si LVM = lvm-vg (<code>vgs</code>) / sinon absent	absent (plain partitions)	collecte si Storage Spaces = storage-pool (<code>Get-StoragePool</code>) / sinon absent (basic disk)
group_name, group_kind	collecte (<code>lsvg</code> , kind=lvm-vg)	collecte (<code>vgs -o vg_name</code> , kind=lvm-vg)	collecte (<code>vgs</code> , kind=lvm-vg)	collecte si LVM (<code>vgs</code>)	absent	collecte (<code>Get-StoragePool.FriendlyName</code> , kind=storage-pool)
size_total_bytes, size_free_bytes	collecte (<code>lsvg</code> : TOTAL/FREE PPs × PP size)	collecte (<code>vgs --units b -o vg_size,vg_free</code>)	collecte (<code>vgs --units b</code>)	collecte si LVM (<code>vgs --units b</code>)	absent	collecte (<code>Get-StoragePool Size/AllocatedSize</code>)
member_disk_count	collecte (<code>lsvg -p</code> / ACTIVE PVs)	collecte (<code>vgs -o pv_count</code>)	collecte (<code>vgs -o pv_count</code>)	collecte si LVM (<code>vgs -o pv_count</code>)	absent	collecte (<code>Get-PhysicalDisk</code> du pool)
state	collecte (<code>lsvg</code> VG STATE)	collecte (<code>vgs -o vg_attr</code>)	collecte (<code>vgs -o vg_attr</code>)	collecte si LVM (<code>vgs</code>)	absent	collecte (<code>Get-StoragePool.HealthStatus/OperationalStatus</code>)
extent_size_bytes	collecte = PP size (<code>lsvg</code> PP SIZE)	collecte = PE size (<code>vgs -o vg_extent_size --units b</code>)	collecte = PE size (<code>vgs --units b</code>)	collecte si LVM (<code>vgs</code>)	absent	nullable (N/A) — pas d'extents sur Storage Spaces

Champ	AIX (jfs2)	RHEL/Oracle (xfs)	Debian/Ubuntu (ext4)	SUSE/SLES (btrfs)	Linux sans LVM	Windows (NTFS/ReFS)
capabilities.max_lvs, max_pps, vg_type	collecte (<code>lsvg</code> : MAX LVs, MAX PPs, VG type big/scalable)	nullable (N/A) — concepts AIX	nullable (N/A) — concepts AIX	nullable (N/A) — concepts AIX	absent (pas de groupes)	nullable (N/A) — concepts AIX
capabilities.resiliency (Windows)	nullable (N/A)	nullable (N/A)	nullable (N/A)	nullable (N/A)	absent (pas de groupes)	collecte (<code>Get-StoragePool</code> / <code>Get-ResiliencySetting</code> : mirror/parity/simple)
<i>storage_topology.volumes[]</i>						
volumes[] (niveau)	collecte = lvm-lv (<code>lslv</code>)	collecte si LVM = lvm-lv (<code>lvs</code>) / partition sinon	collecte si LVM = lvm-lv (<code>lvs</code>) / partition sinon	collecte si LVM = lvm-lv (<code>lvs</code>)	collecte = partition (<code>lsblk</code>) / groupe vide	collecte = virtual-disk (Storage Spaces) ou partition (basic disk)
volume_name, volume_kind	collecte (<code>lslv</code> , kind=lvm-lv)	collecte (<code>lvs -o lv_name</code> , kind=lvm-lv)	collecte (<code>lvs</code> , kind=lvm-lv)	collecte si LVM (<code>lvs</code>)	collecte (<code>lsblk -o NAME</code> , kind=partition)	collecte (<code>Get-VirtualDisk.FriendlyName</code> kind=virtual-disk <code>Get-Partition</code> kind=partition)
group_ref	collecte = rootvg/datavg (<code>lslv</code>)	collecte = VG (<code>lvs -o vg_name</code>)	collecte = VG (<code>lvs -o vg_name</code>)	collecte si LVM (<code>lvs</code>)	nullable (N/A) — pas de VG	collecte = pool ref (<code>Get-VirtualDisk</code>) / nullable si basic disk
size_bytes	collecte (<code>lslv</code> : LPs × PP size)	collecte (<code>lvs --units b -o lv_size</code>)	collecte (<code>lvs --units b</code>)	collecte si LVM (<code>lvs --units b</code>)	collecte (<code>lsblk -b -o SIZE</code>)	collecte (<code>Get-VirtualDisk.Size</code> / <code>Get-Partition.Size</code>)
type	collecte (<code>lslv</code> TYPE jfs2/jfs2log/paging)	collecte (linear/thin, <code>lvs -o lv_layout</code>)	collecte (linear/thin, <code>lvs -o lv_layout</code>)	collecte si LVM (<code>lvs</code>)	collecte (type partition, <code>lsblk -o PARTTYPE</code>)	collecte (<code>Get-VirtualDisk</code> / type partition GPT)
redundancy	collecte (<code>lslv</code> COPIES = mirroring)	collecte (mirror/raid si lvmraid, <code>lvs -o lv_attr</code>)	collecte (<code>lvs -o lv_attr</code>)	collecte (profil RAID btrfs si pertinent / <code>lvs</code>)	nullable (N/A) — partition simple	collecte = resiliency (<code>Get-VirtualDisk.ResiliencySettingName</code> mirror/parity/simple)
<i>storage_topology.disks[]</i>						

Champ	AIX (jfs2)	RHEL/Oracle (xfs)	Debian/Ubuntu (ext4)	SUSE/SLES (btrfs)	Linux sans LVM	Windows (NTFS/ReFS)
disk_name	collecte (<code>lspv</code> hdiskN)	collecte (<code>pvs -o pv_name / lsblk</code>)	collecte (<code>pvs / lsblk</code>)	collecte (<code>pvs / lsblk</code>)	collecte (<code>lsblk -d -o NAME</code>)	collecte (<code>Get-Disk.Number/FriendlyName</code> , <code>Get-PhysicalDisk</code>)
size_bytes	collecte (<code>lspv / bootinfo -s hdiskN × 1024</code>)	collecte (<code>pvs --units b / lsblk -b -d -o SIZE</code>)	collecte (<code>pvs --units b / lsblk -b</code>)	collecte (<code>lsblk -b -d</code>)	collecte (<code>lsblk -b -d -o SIZE</code>)	collecte (<code>Get-Disk.Size / Get-PhysicalDisk.Size</code>)
free_bytes	collecte (<code>lspv</code> FREE PPs × PP size)	collecte si LVM (<code>pvs -o pv_free --units b</code>) / nullable sinon	collecte si LVM (<code>pvs --units b</code>) / nullable sinon	collecte si LVM (<code>pvs</code>) / nullable sinon	nullable (N/A) — pas de notion VG-free	collecte (<code>Get-PhysicalDisk</code> espace non alloué du pool) / nullable si basic disk
group_ref	collecte = rootvg/datavg (<code>lspv</code>)	collecte = VG si LVM (<code>pvs -o vg_name</code>) / nullable sinon	collecte = VG si LVM (<code>pvs</code>) / nullable sinon	collecte = VG si LVM / nullable sinon	nullable (N/A) — pas de VG	collecte = pool si Storage Spaces (<code>Get-PhysicalDisk -> pool</code>) / nullable si basic
media_type	collecte (<code>lsdev</code> , <code>lsmPIO</code> / SSD-HDD si expose)	collecte (<code>lsblk -d -o ROTA -> ssd/hdd</code>)	collecte (<code>lsblk -d -o ROTA</code>)	collecte (<code>lsblk -d -o ROTA</code>)	collecte (<code>lsblk -d -o ROTA</code>)	collecte (<code>Get-PhysicalDisk.MediaType</code> SSD/HDD/Unspecified)
<i>paging[]</i>						
name, kind	collecte = aix-paging-lv (<code>lsps -a</code>)	collecte = linux-swap-partition linux-swap-file (<code>swapon --show</code>)	collecte = linux-swap-partition linux-swap-file (<code>swapon --show</code>)	collecte (<code>swapon --show</code> ; zswap/swapfile possibles)	collecte (<code>swapon --show</code>)	collecte = windows-pagefile (<code>Win32_PageFileUsage</code>)
size_total_bytes, size_used_bytes	collecte (<code>lsps -a</code> : Size Mo × %Used)	collecte (<code>swapon --show=SIZE,USED --bytes</code>)	collecte (<code>swapon --show=SIZE,USED --bytes</code>)	collecte (<code>swapon --show --bytes</code>)	collecte (<code>swapon --show --bytes</code>)	collecte (<code>Win32_PageFileUsage AllocatedBaseSize/CurrentUsage</code> Mo -> octets)

Regles de branchement Ansible

Le module de collecte branche d'abord sur `ansible_facts['system']` / `platform` , puis sur le fstype par filesystem. Une seule tâche logique par OS, sorties normalisées vers le schéma v2 (octets partout).

- **Detection initiale** : `gather_facts` (ou `setup`) -> `ansible_system` ∈ { AIX, Linux, Win32NT }. Le `connection` est déjà choisi par l'inventaire : `ssh` pour AIX/Linux, `winrm` (ou `psrp`) pour Windows.
- **Branche AIX** (`when: ansible_system == "AIX"`) : connexion SSH. Topologie LVM via `lsvg`, `lsvg -l <vg>`, `lsvg -p <vg>`, `lslv <lv>`, `lspv` ; filesystems via `df -k` + `lsfs -c` (jfs2) ; paging via `lspv -a`. `reserved_bytes` force a null, `acl_summary` null, `capabilities.max_lvs/max_pps/vg_type` renseignés. Conversions Ko/PP -> octets cote parsing.
- **Branche Linux** (`when: ansible_system == "Linux"`), puis sous-branche par `fstype` (issu de `findmnt -no FSTYPE` par mount) :
 - **xfs (RHEL/Oracle/Rocky/Alma)** : `df -B1` + `df -i` + `xfs_info` ; `reserved_bytes` = null. `/boot/efi` collecte seulement si UEFI x86_64 (`test -d /sys/firmware/efi`).
 - **ext4 (Debian/Ubuntu)** : `df -B1` + `df -i` + `tune2fs -l` pour `reserved_bytes` (Reserved block count × Block size).
 - **btrfs (SUSE/SLES)** : NE PAS se fier a `df` ; tailles vraies via `btrfs filesystem usage <mp>` et `btrfs qgroup show` (sous-volumes + snapshots) ; `reserved_bytes` = null.
 - **Sous-branche LVM** (orthogonale au fstype, `when: lvs présent`) : `vgs/lvs/pvs --units b --nosuffix` -> groups/volumes/disks. Si pas de LVM (plain partitions) : `storage_topology.groups` absent, `volumes[]` = partitions (`lsblk -b`), `group_ref` null.
 - **paging** : `swapon --show --bytes` (distingue partition vs fichier swap).
- **Branche Windows** (`when: ansible_os_family == "Windows"`) : connexion WinRM/PSRP, modules `ansible.windows.win_powershell` / `win_shell`. Inventaire PowerShell : `Get-Volume`, `Get-Partition`, `Get-Disk`, `Get-PhysicalDisk`, `Get-StoragePool`, `Get-VirtualDisk`, `Win32_PageFileUsage`, `Get-Acl` (resume ACL non sensible). Mapping : pool -> `groups[]` (storage-pool), virtual disk -> `volumes[]` (virtual-disk), basic disk -> partitions en `volumes[]` avec `groups` absent. `inodes_*` = null, `mount_options` = null (remplace par `acl_summary`, `access_model=ntfs-acl`), `reserved_bytes` = null. Tailles déjà en octets natifs. Volume systeme (BootDevice / emplacement de Windows) -> `storage_class=system`, autres -> `data`.
- **Derivation post-collecte (commune)** : lien filesystems <-> topologie par clefs `group_ref` / `backing_ref` ; `storage_class` (system vs data) derive du `group_ref` (rootvg / C: / volume de boot = system, datavg / autres = data). Aucune branche n'omet un champ obligatoire : les champs OS-spezifiques sont mis a `null`, jamais supprimés.

Principe nullable vs absent

Deux mecanismes distincts, a ne pas confondre :

- **nullable (N/A)** : la clef EST presente dans le document JSON, avec la valeur `null` . On l'emploie quand le champ appartient au niveau de denominateur universel ou a un objet effectivement instancie, mais que la valeur n'a pas de sens pour cette combinaison OS/fstype. Exemples :
`reserved_bytes` = null sur xfs/jfs2/btrfs/ntfs (seul ext4 le renseigne) ; `inodes_total/used` = null sur Windows ; `acl_summary` = null sur Unix ;
`mount_options` = null sur Windows ; `capabilities.max_lvs/max_pps/vg_type` = null hors AIX ; `extent_size_bytes` = null sur Storage Spaces ;
`group_ref / free_bytes` = null sur disque/partition hors LVM. Garder la clef preserve un schema stable, colonne-compatible, et permet de DISTINGUER "non applicable" de "non collecte".
- **absent** : la clef (ou le niveau entier de `storage_topology`) N'EST PAS emise, parce que la structure n'existe pas sur l'hote. C'est le cas des NIVEAUX de topologie vides : `storage_topology.groups` est absent (tableau vide) sur du Linux en partitions simples sans LVM et sur du Windows en disques basiques (pas de Storage Spaces). On ne fabrique pas de groupe fictif ; les filesystems pointent alors directement sur des `volumes[]` de type partition, et `group_ref` vaut null.

Regle d'or : un champ OBLIGATOIRE n'est jamais silencieusement supprime. S'il ne s'applique pas, il devient `null` (nullable) et reste auditable. Seuls les NIVEAUX de topologie structurellement inexistants sont `absent` (collection vide), ce qui se lit comme "cet hote n'a pas cette couche d'abstraction de stockage" et non comme "donnee manquante". Cette discipline garantit que l'ensemble obligatoire couvre les 5 objectifs sur chaque OS, tout en gardant les champs specifiques OS/fstype nullable plutot que perdus.